西 安 邮 电 大 学

AI 全栈开发实验

题目：前后端开发实验

组　　号：23

组员信息：

学号	姓名	分工
2610321101	何朝杨	后端 AI 服务、架构设计文档与成员分工表
2610321103	杜军博	报告整合、排版与正式 PDF；核对推理耗时口径
2610321135	王陆洲	API 文档与测试报告、模型产物上传
2610320046	张梓萱	前端页面架构与公共组件、响应式与浏览器兼容
2610320033	刘凡思雨	前端三页面交互与数据流、端到端与性能测试
2610321168	秦鑫洋	问题与解决方案、截图录屏与交付材料整理

时　　间：2026.11.05

一、实验目的

本实验承接实验 1 的电商商品图文数据、实验 2 微调的 Chinese-CLIP[1] 中文图文检索双塔模型与实验 3 的 ONNX[2] 推理加速产物，把模型能力封装为可用的 Web 应用，形成“数据—模型—应用”的完整闭环，具体目的如下：

1. 构建前后端完整架构：前端提供图像上传、中文查询与图文比对三类交互界面，后端以 RESTful[3] API 封装模型推理，使实验 1—3 的成果第一次面向真实用户暴露；
2. 掌握前端 AI 交互界面开发：基于 React[4] 与 Ant Design[5] 实现适配实验 1 图像（商品图）与文本（中文查询词、商品文案）两类模态的交互界面，覆盖上传、输入、前端校验、结果展示与异常提示的完整链路；
3. 完成后端模型服务化：加载实验 3 导出的双塔 ONNX 产物，把“图 → 向量 → 相似检索”与“文 → 向量 → 相似检索”封装为稳定接口，实现模型推理的服务化，并复用实验 1 的图像预处理口径；
4. 掌握前后端数据流设计与 API 规范：统一返回体、统一错误码、请求格式分离（表单 vs JSON），把任务书要求的“400 参数错 / 404 路径错 / 500 服务器错”落实到具体场景；
5. 解决跨平台兼容与分布式任务协调：以响应式布局覆盖手机 / 平板 / PC，以 Celery[6] + Redis 把批量与大图推理从同步请求中剥离，避免长任务阻塞交互；
6. 形成可复现的工程交付：接口文档由代码自动生成，测试用例与截图可复跑，报告中每个数字都可追溯到具体的命令与运行环境。

二、实验环境

2.1 软硬件环境

本实验的全部实测在统一的软硬件环境中完成，环境明细见表 1。表中每一项均在报告中用于解释某个数字的来源，避免“同一指标混用不同环境”。

项目	配置与版本
硬件	通用 PC（8 逻辑核），Windows 11 x64；后端与浏览器同机运行
后端运行时	Python 3.12.13 标准 venv（`webapp/backend/.venv`），不依赖 Anaconda
Python 依赖	FastAPI[7] 0.162.0、Uvicorn 0.54.0、Pydantic 2.14.2、NumPy 2.5.4、Pillow 13.0.0、Celery 5.6.3、redis 8.1.0、onnxruntime 1.31.1
前端运行时	Node.js 24.19.0、npm 11.17.0；Vite[8] 5.4.21、React 19、Ant Design 5.27、Axios 1.7、React Router 7、Vitest 2.1
模型	实验 3 导出的 Chinese-CLIP 双塔 ONNX（FP32）：图像塔 345.16 MB、文本塔 408.50 MB，合计约 754 MB，SHA-256 校验通过
检索索引	`data/index_data_finetuned/`，9 类共 9,393 条、512 维，由实验 2 的 `epoch3.pt` 编码
浏览器	Chrome 156（Chromium 内核）为功能与截图主环境；Edge 同内核复核
报告工具链	Typst 0.15.1 + CeTZ 0.3.4，后端质量门禁 black / ruff / mypy / unittest

表 1 实验软硬件环境

需特别说明两点口径。其一，本实验的模型加载在 **CPU** 上以 ONNX Runtime 的 CPUExecutionProvider 完成，未使用 GPU 与 TensorRT，因此下文所有延迟数字均属 CPU 口径，不可与实验 3 的 GPU 测速结果直接比较。其二，模型产物体积约 754 MB，不在版本库中，由 GitHub Release 分发并通过 `scripts/fetch_model_assets.py` 下载、校验 SHA-256 后落盘，仓库对 `data/optimize/` 整体忽略。

2.2 环境搭建过程

后端与前端的环境搭建命令见代码 1。二者互不依赖，可并行进行。

```
# 后端
cd webapp\backend
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
Copy-Item .env.example .env
pip install onnxruntime                          # 仅在接入真模型时需要
python -m uvicorn app.main:app --host 127.0.0.1 --port 8001

# 前端
cd webapp\frontend
npm install
npx vite --port 5173
```

代码 1 后端虚拟环境与前端依赖安装

搭建过程中遇到并解决的环境问题见表 2，其中前两条会直接导致“启动失败”，后两条会导致“看起来通了但结果不对”，是最容易浪费时间的两类问题。

序	现象	原因	处理
1	`cannot import name 'Self' from 'typing'`	后端使用 `typing.Self`，该符号要求 Python ≥ 3.11，本机默认 3.10.11	改用 Python 3.12.13 重建 venv；建议在 `pyproject.toml` 显式声明 `requires-python`
2	`WinError 10048` 端口被占用	8000 端口已有其他进程监听	后端改用 8001；前端代理目标必须同步修改，否则“连上了但不是这个后端”
3	前端代理始终返回 `{\"detail\":\"Not Found\"}`	`vite.config.js` 中代理目标写死为 8000，环境变量不生效	临时以不入库的 `vite.config.local.mjs` 覆盖；长期应把代理目标参数化
4	ONNX 报 `ModuleNotFoundError: torch`	官方 `cn_clip` 包在顶层 `import torch` 与 `torchvision`，而推理只需其中的分词与图像变换	仓库内提供等价的 torch-free 实现（见 3.3.2），避免为此安装约 2 GB 依赖

表 2 环境搭建中的典型问题与处理

三、实验内容与步骤

任务书把本实验拆为阶段 1 前端开发（4.1）与阶段 2 后端开发与集成（4.2）。本节依次给出需求分析、设计思路、代码实现与测试方法，四个小节与任务书“1．需求分析 / 2．设计思路 / 3．代码实现 / 4．测试优化”一一对应。

3.1 需求分析

3.1.1 前端需求

前端的核心是“三类交互、两类模态、一套通用能力”：

1. 三类交互对应任务书的三个页面：图像分类页（上传商品图 → 判类别 → 推荐相似商品）、文本检索页（输入中文查询 → 返回相关商品）、多模态匹配页（同时给图与文 → 判两者是否描述同一类商品）；
2. 两类模态对应实验 1 的数据：图像为 PNG/JPG/WEBP、单张 ≤ 10 MB（与后端上限一致，前端先拦一道，避免大文件无谓传输）；文本为中文查询词，长度 1–200 字；
3. 通用能力包括导航与路由、上传区（拖拽 + 点击 + 示例图）、结果展示区（类别标签、置信度、相似商品卡片）、异常提示（网络错误、超时、业务错误码）；
4. 跨平台要求手机、平板、PC 三档视口均不出现横向溢出，表格在窄屏可横向滚动而非压缩列宽。

3.1.2 后端需求

1. 模型服务化：以进程内单例加载双塔编码器，启动时完成一次加载（实测两塔各约 1.4 s），请求期不再重复初始化；
2. 检索服务化：图像与文本两路向量与预建索引做余弦相似度检索；图像分类需给出各类最高分并按 softmax 折算置信度；
3. 工作流编排：图文匹配必须“图像独立判类 → 文本独立判类 → 比对一致性”，不能用图像判出的类别去限定文本检索，否则比对恒为一致、功能退化；
4. 异步协调：批量与大图推理走任务队列，接口立即返回 `task_id`，前端轮询取结果；同步接口不依赖队列，保证队列故障时核心功能仍可用；
5. 数据入库：支持上传新商品并增量写入索引，写入后无需重启即可被检索到；
6. 契约稳定：统一返回体 `{\"code\", \"result\", \"msg\"}`、统一错误码、RESTful 名词路径加版本前缀、接口文档自动生成。

3.1.3 非功能需求

表 3 给出四项非功能需求及其验收方式。其中响应时间是任务书给出的硬指标，本报告在 5.3 节的表 13 以实测数据判定。

需求	含义	验收方式
响应时间	前端从提交到展示结果的延迟 ≤ 2 s	表 13 端到端实测
可维护性	路由只做取参与校验、服务层只做编排、数据访问层只管索引读写	代码分层审查
可测试性	编码器与索引均为可替换接缝，测试不依赖 GPU 与模型文件	74 项后端自检 + 21 项前端单测
可复现性	接口文档由代码生成，测试脚本可重跑	`openapi.json` + 测试脚本

表 3 非功能需求与验收方式

3.2 设计思路

3.2.1 总体架构

系统采用“浏览器单页应用 + RESTful 后端服务 + 进程内模型 + 只读检索索引”的四层结构。整体链路自左向右为“浏览器 → Vite 代理 → 后端”，后端内部再自“接入层”向下依次落到服务层与能力层，最终由底部的模型产物与索引数据支撑，如图 1 所示。

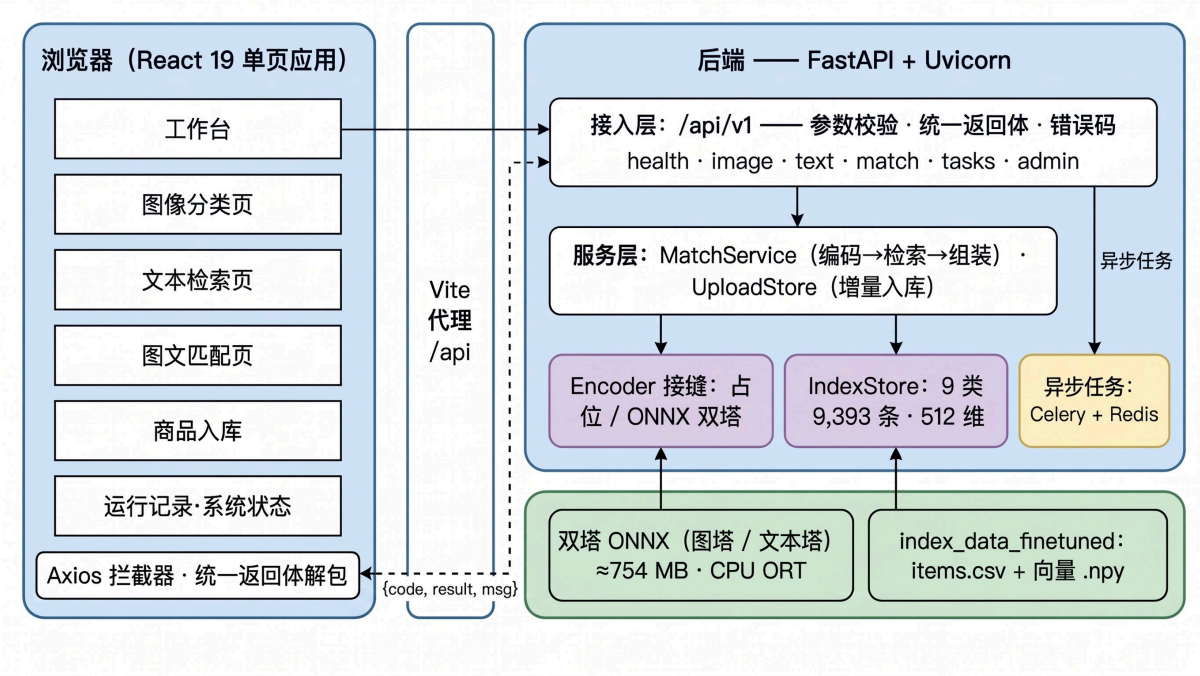


图 1 前后端总体架构与数据流

各层职责如下：

1. 表现层（React）：6 个路由页面加公共组件；所有请求经统一的 Axios 实例发出，响应拦截器把 `{\"code\", \"result\", \"msg\"}` 拆解为“成功取 result / 失败抛 AppError”，页面只处理业务数据与错误提示；
2. 接入层（FastAPI 路由）：`/api/v1` 下 9 个端点，按页面分组打 tag（状态、图像分类页、文本检索页、多模态匹配页、数据入库）；路由函数只做参数校验、调用服务、包装返回体；

3. 服务层：`MatchService` 编排“编码 → 检索 → 组装”，不感知 HTTP；`UploadStore` 负责增量入库；Celery 任务复用同一服务层；
4. 能力层：`Encoder` 与 `IndexStore` 构成两个可替换接缝；`IndexStore` 按类别懒加载并缓存，避免启动时全量读盘；
5. 异步任务（Celery + Redis）：旁挂在接入层之下，承接批量与大图请求，复用服务层逻辑但不阻塞同步接口；
6. 数据底座：底部的双塔 ONNX 产物（约 754 MB，CPU ORT 运行）与 `index_data_finetuned`（9 类 9,393 条、512 维）分别向 `Encoder` 与 `IndexStore` 提供输入，二者必须同源，否则在检索前会因维度校验失败而报 `40003`。

3.2.2 关键设计决策

1. **统一返回体而非 HTTP 语义直出**：任务书要求 `{\"code\", \"result\", \"msg\"}` 且要求 400/404/500 三档。本实现让业务码前三位等于 HTTP 状态码（`40001 → 400`），既满足任务书又保留细分场景；
2. **编码器采用两档同形接缝**：`PlaceholderEncoder` 与 `OnnxEncoder` 对外接口完全一致，切换只改 `.env` 的 `SHOPVISION_MODEL_BACKEND`。占位实现用输入内容的 SHA-256 派生随机种子，保证“同输入同向量”，使前端与后端可在模型未就位时并行开工；
3. **检索只算查询侧**：索引侧 9,393 条向量在建索引时已算好并 L2 归一化，线上只需编码用户输入的一个样本，相似度退化为一次矩阵乘法，开销集中在模型前向；
4. **图文匹配不做以图限文**：两条路径独立编码、独立判类后再比对，避免循环论证，这是与“先分类再检索”最本质的区别；
5. **错误码单一来源**：错误码定义为 `IntEnum`，HTTP 状态由 `code // 100` 推导，文档、异常处理与前端兜底共用同一张表，避免三处各写一份而漂移。

3.3 代码实现

3.3.1 前端页面与组件

前端按职责切分：`pages/` 放路由页，`components/` 放可复用区块，`context/` 放跨页状态，`api/client.js` 放统一请求封装。路由、页面职责与所调接口的对应关系见表 4。

路由	页面	职责	接口
/	工作台	搜索横幅、指标栏、9 类照片轮播、最近运行	GET /health
/image	图像分类	拖拽上传 / 示例图、置信度结果卡、Top3 候选、批量连续测试	POST /image
/text	文本检索	中文查询 + 类别限定 + top_k，返回商品卡片列表	POST /text
/match	图文匹配	图 / 文两路独立判类、一致性结论、检索证据	POST /match
/admin	商品入库	上传商品、新建类别、增量写入索引	POST /admin/items
/history	运行记录	localStorage 持久化的运行轨迹，可重放与导出	—
/status	系统状态	模型后端、索引明细、接口契约与错误码表	GET /health

表 4 前端路由、页面职责与所调接口

统一请求封装是前端“薄”的关键，见代码 2。响应拦截器把三种情况归一到一处：业务码非 0、HTTP 非 2xx、网络或超时，页面因此不必各自判断。

```
client.interceptors.response.use(
  (response) => {
    const body = response.data
    if (!body || typeof body.code !== 'number') {
      throw new AppError('INVALID_RESPONSE', '后端返回格式异常', response.status)
    }
    if (body.code !== 0) throw new AppError(body.code, body.msg || '请求失败',
response.status)
    return body.result          // 页面只拿 result
  },
  (error) => {
    if (error.code === 'ECONNABORTED') throw new AppError('TIMEOUT', '请求超时，图片可能过
大')
    if (!error.response) throw new AppError('NETWORK', '连不上后端服务')
    const body = error.response.data
    throw new AppError(body?.code ?? error.response.status, body?.msg || '请求失败')
  },
)
```

代码 2 Axios 响应拦截：把统一返回体与传输层错误归一为 AppError

3.3.2 后端分层与模型接入

后端分为 api/（路由）、services/（编排与能力）、schemas.py（数据契约）、errors.py（错误码）、deps.py（依赖注入）。依赖注入采用“单例 + 可覆盖”的组合，见代码 3。

```python
@lru_cache(maxsize=1)
def get_encoder() -> Encoder:
    return build_encoder(get_settings())        # placeholder 或 onnx，由 .env 决定

@lru_cache(maxsize=1)
def get_index() -> IndexStore:
    settings = get_settings()
    return IndexStore(settings.index_path, candidate_k=settings.candidate_k)

def get_match_service() -> MatchService:
    return MatchService(get_encoder(), get_index())   # 路由只写
Depends(get_match_service)
```

代码 3 编码器与索引的单例装配（测试可整体覆盖）

模型接入的真实形态是“进程内加载两个 ONNX 会话，预处理与分词在图外完成”，见代码 4。此处有一个工程细节值得记录：官方 `cn_clip` 包在顶层即 `import torch`，而推理真正需要的只有分词器与图像变换；为此本仓库提供了一个逐行对齐官方实现、但不依赖 torch 的等价实现，使接入真模型无须安装约 2 GB 的深度学习框架。

```python
self._visual = ort.InferenceSession(str(visual_path), providers=[provider])
self._text   = ort.InferenceSession(str(text_path),   providers=[provider])
self._preprocess = image_transform(image_size)     # 224x224、CLIP 均值方差

def encode_image(self, image):
    arr = self._preprocess(image.convert("RGB")).numpy()[None].astype("float32")
    return _l2(self._visual.run(None, {"image": arr})[0][0])

def encode_text(self, text):
    ids  = self._tokenize([text], context_length=self.context_length)   # CLS+正文+SEP
    mask = ids.ne(0).long()
    return _l2(self._text.run(None, {
        "input_ids": ids.numpy().astype("int64"),
        "attention_mask": mask.numpy().astype("int64"),
    })[0][0])
```

代码 4 ONNX 双塔编码器：预处理、分词与归一化均在计算图之外完成

3.3.3 图文匹配工作流

图文匹配是本实验“多模型协同工作流”的落点。任务书描述为“先图像分类 → 再文本分类 → 比对类别一致性”。本实现在语义上严格执行，但不把图像类别作为文本检索的过滤条件，实现见代码 5。

```python
    def match(self, image, text, top_k=5):
        image_vector = self.encoder.encode_image(image)
        text_vector  = self.encoder.encode_text(text)
        image_category, _, _ = self.index.best_category(image_vector)    # 图像塔独立判类
        text_category,  _, _ = self.index.best_category(text_vector)     # 文本塔独立判类
        hits = self.index.search(text_vector, top_k=top_k)               # 证据：不限类别
        return {
            "match": bool(image_category) and image_category == text_category,
            "image_category": image_category,
            "text_category": text_category,
            "items": [hit.as_dict() for hit in hits],
        }
```

代码 5 图文匹配：两路独立编码、独立判类，最后只做一致性比对

若改为“先用图像类别限定文本检索”，则 `text_category` 恒等于 `image_category`，“是否匹配”这一结论将永远为真。因此保留全类别检索结果，既作为证据，也便于演示“不一致”场景。

3.3.4 异步任务编排

同步接口一次只能处理一张图，批量或大图会显著拉长请求。为此引入 Celery + Redis：提交接口只做校验并入队，立即返回 `task_id`；前端按约 800 ms 轮询状态。状态机见表 5。

status	含义	前端动作
PENDING	排队中；不存在的 task_id 亦返回此值，与 Celery 语义一致	继续轮询
PROGRESS	执行中，progress 给出已完成 / 总数	继续轮询
SUCCESS	完成，result 为业务数据	停止轮询，渲染结果
FAILURE	失败，error 含业务码与提示	按 msg 提示

表 5 异步任务状态机与前端动作

3.3.5 API 清单

共 9 个端点，按业务分组见表 6。完整参数、返回值结构与调用示例由 `export_api_docs.py` 自动生成于 `docs/交付材料/实验四_第23组/api/`，其中 `openapi.json` 可直接导入 Postman。

方法	路径	分组	说明
GET	/health	状态	探活、模型后端、索引概况与各类条数
POST	/image	图像分类页	表单上传；返回类别、置信度、Top3 与相似商品
POST	/text	文本检索页	JSON 提交；返回相关商品列表与查询判类
POST	/match	多模态匹配页	表单上传；返回两路类别与一致性结论
POST	/tasks/image	状态	提交单图异步分类，返回 task_id
POST	/tasks/batch	状态	提交批量异步分类，单批上限 12 张
GET	/tasks/{task_id}	状态	轮询任务状态与进度
POST	/admin/items	数据入库	上传商品入库，写入后立即可检索
POST	/admin/categories	数据入库	新建空类别，空类别不参与自动判类

表 6 接口清单（基址 /api/v1）

3.3.6 异常处理

错误码定义为单一来源的 IntEnum，业务码前三位即 HTTP 状态，全表见表 7。新增场景时只增新码、不改老码含义，前端按整段码定位、按前三位兜底。

业务码	HTTP	场景	触发点
0	200	成功	—
40000	400	参数校验失败	空查询、top_k 越界、查询词超长、非法类别名
40001	400	图像格式错误或损坏	上传内容不是可视 PNG / JPG / WEBP
40002	400	上传内容为空	file 为零字节
40003	400	向量维度与索引不一致	模型与索引不同源
40004	400	上传文件过大	超过 10 MB 上限
40400	404	路径或资源不存在	访问未注册 URL
40401	404	类别不存在	category 不在索引或上传库中
40402	404	索引文件缺失	注册表 / 条目表 / 向量文件缺失
50000	500	未捕获服务器错误	未预料异常，仅记服务端日志
50001	500	模型或依赖不可用	ONNX 缺失、依赖未装、占位模式禁止入库

表 7 错误码表与触发场景

前端侧的异常处理原则是“msg 直接给用户看”：4xx 提示用户改输入，5xx 提示稍后重试；网络错误与超时单独成码（NETWORK / TIMEOUT），不与业务码混用。

3.4 测试方法

测试分五层，全部可复跑，覆盖单元、契约、端到端、异常与性能。各层的工具、用例规模与判据见表 8。

层次	工具与命令	用例规模	判据
后端单元	`python -m unittest discover -s tests -v`	74 项	全部通过
接口契约	由 `openapi.json` 断言端点、分组、返回体与错误码	9 端点 × 4 类断言	全部通过
前端单元	`npm test`（Vitest）	21 项	全部通过
端到端功能	真实前后端 + Chrome 驱动交互	32 条用例	逐条比对预期
性能	HTTP 客户端预热后计时 10 次	4 个接口	中位数 ≤ 2 s

表 8 测试分层、工具与判据

端到端功能测试不满足于“接口能返回 200”，而是为每条用例预设了期望值：图像用例以样例图文件名所代表的类别为真值；文本用例以查询词的语义类别为真值；匹配用例分别构造“应当一致”与“应当不一致”两组。这样才能区分“链路通”与“结果对”，两者的差别在 5.2 中有直接体现。

四、实验关键技术与难点

4.1 关键技术

4.1.1 统一返回体与错误码分层

任务书要求“400 参数错 / 404 路径错 / 500 服务器错”，但工程上仅这三档无法区分“图片格式错”与“图片太大”。本实现让业务码为 5 位、前三位等于 HTTP 状态码：`40001` 既映射为 HTTP 400，又精确表达“图像格式错误”。该设计的关键是把错误码做成**唯一来源**（`IntEnum` 加 `code // 100` 推导 HTTP 状态），使接口文档、后端异常处理与前端提示三处不会各写一份而漂移。表 7 与后端代码共用同一张表，因此不存在“文档说 404、实现返回 200”的可能。

4.1.2 编码器接缝与两档同形实现

`Encoder` 是抽象基类，`PlaceholderEncoder` 与 `OnnxEncoder` 对外接口完全一致。这一设计的价值在本实验中有直接体现：模型产物（754 MB）到位之前，前后端可以先用占位编码器把接口、页面、错误处理全部做完；产物到位后只改一行配置即切换到真实推理，接口形状、前端代码、测试用例一行未动。换言之，“模型”从开发期依赖变成了部署期依赖，这是本实验能在有限学时内完成的根本原因。

4.1.3 检索侧只算一次

索引侧 9,393 条 512 维向量在实验 2 建索引时已算好并 L2 归一化，因此“余弦相似度”等价于点积，线上一次 `embeddings @ query` 即完成全库比对，再取 Top-K。真正的耗时集中在模型前向——这与 5.3 的实测一致：文本检索端到端约 71 ms，其中向量比对部分不足 1 ms，其余几乎全是文本塔前向与序列化开销。这也正是实验 3 引入推理加速的直接动因，两个实验在此闭环。

4.1.4 无 torch 的 cn_clip 等价实现

官方 cn_clip 包在 __init__ 顶层即加载 torch 与 torchvision，而用 ONNX 推理时真正需要的只有两件事：把中文切成 [CLS] 正文 [SEP] 的 id 序列，以及把图像缩放、归一化成 3×224×224 的浮点张量。本仓库据此实现了一份等价代码，逐行对齐官方 utils.py 的口径（context_length=52、BICUBIC、CLIP 的均值与方差），对外提供同名同参接口，返回值以 NumPy 数组代替 torch 张量。如此既保证与官方完全一致的数值口径，又省去约 2 GB 依赖，并让后端可以在一个纯 CPU 的轻量虚拟环境中复现。

4.1.5 依赖注入与单例

编码器与推理会话的加载成本高（实测两塔各约 1.4 s、合计占内存数百 MB），必须进程内单例；但单例又会阻碍测试与替换。Depends 加 lru_cache 的组合同时满足两者：线上只加载一次，测试用 dependency_overrides 整体替换，无需改动任何业务代码。

4.1.6 异步任务与同步接口解耦

Celery 只承接“批量”与“大图”两类长任务，三个同步查询接口不依赖 Redis。这样即使队列不可达，系统的核心演示功能仍完整可用，降级边界清晰；队列故障时只有异步提交接口返回 50001，其提示直接说明应检查 broker 地址。

4.2 主要难点与解决

1. **图文匹配的循环论证陷阱**：若用图像类别去限定文本检索，则比对恒为一致，功能退化。解决方法是两路独立编码、独立判类，只在最后比对，并把全类别检索结果作为证据返回；表 11 中的“不一致”用例正是对这一设计的检验；
2. **ONNX 图外的前后处理**：导出后 ONNX 只接受 input_ids、attention_mask 与归一化图像张量，分词器与 L2 归一化均须在代码中补齐，否则向量不可比。分词与图像变换的每个参数（截断长度、插值方式、均值方差）都必须与训练时一致，任何一处偏差都会让相似度整体漂移；
3. **模型与索引必须同源**：编码器维度与索引维度不一致时，矩阵乘法会静默算出错误的相似度而不报错。解决方法是检索前显式校验维度并抛 40003，把静默错误变成可诊断错误；
4. **环境版本耦合**：typing.Self 要求 Python ≥ 3.11；端口冲突需同时改监听端口与前端代理目标；官方 cn_clip 引入不必要的重依赖。三者在表 2 中均已记录；
5. **界面与数据解耦**：指标栏、类别名、商品图全部来自 /health 与接口返回，前端不留写死的演示数据，避免“截图漂亮但数据是假的”。

五、实验结果与分析

本章全部数据在 2026-11-05 的真实 ONNX 双塔接入状态下测得，测试脚本与结构化结果分别归档于 docs/交付材料/实验四_第 23 组/测试/。凡涉及模型判断的结果，均以“是否判对”而非“是否返回 200”为评价依据。

5.1 自动化测试结果

5.1.1 后端自检

后端 74 项自检覆盖索引读写、服务编排、上传入库、批量导入与 OpenAPI 契约，结果见代码 6。该套测试不依赖模型文件与 GPU，因此可在任何机器上复跑。

```
$ cd webapp/backend && .\.venv\Scripts\python.exe -m unittest discover -s tests -v
.....................................................................
Ran 74 tests in 2.935s
OK
```

代码 6 后端自检运行结果（Python 3.12.13 venv）

5.1.2 前端单元测试

前端 21 项单元测试覆盖请求封装、错误转换、商品图地址解析与上传文件校验，结果见代码 7。四个测试文件分别对应一类“容易在重构中悄悄坏掉”的逻辑，其中上传校验的用例直接对应任务书“图像格式错误，请上传 PNG/JPG”这一要求的前端实现。

```
$ npm test
 ✓ src/__tests__/utils/errors.test.js        (3 tests)
 ✓ src/__tests__/utils/itemImage.test.js     (11 tests)
 ✓ src/__tests__/utils/validateImageFile.test.js (5 tests)
 ✓ src/__tests__/api/client.test.js          (2 tests)

 Test Files  4 passed (4)
      Tests  21 passed (21)
```

代码 7 前端单元测试运行结果（Vitest）

5.1.3 测试环境与口径

为避免“同一数字在不同环境下不可比”，本报告固定以下口径：模型为 CPU 上的 ONNX Runtime FP32、8 线程、图优化全开；索引为 9 类 9,393 条；测试数据为 webapp/frontend/public/samples/ 下 9 张分类样例图；延迟为端到端 HTTP 往返时间（含解码、预处理、前向、检索与 JSON 序列化），预热 2 次后连续计时 10 次取中位数。该口径与实验 3 的“仅模型 forward、GPU TensorRT”口径**不同**，两者数字不可横向比较，这一点在 5.3.3 会再次强调。

5.2 功能测试结果

5.2.1 图像分类

对 9 张分类样例图逐一提交 `/image`，以文件名所代表的类别为真值比对，结果见表 9。9 条用例全部判对，说明“编码器接缝切换 + 索引同源”这条链路在真实权重下确实成立。

序	样例	期望	实得	相似度	首条相似商品
1	cat-shoe	鞋	鞋	0.957	PUMA CARE OF 男士皮质低帮运动鞋，白色
2	cat-bag	包	包	0.904	香港真皮双肩背包女
3	cat-bottle	水杯	水杯	0.918	水杯科技
4	cat-phonecase	手机壳	手机壳	0.839	Solimo 3D 打印硬壳手机壳
5	cat-shirt	男装	男装	0.899	假两件小 V 领短袖男 T 恤
6	cat-dress	女装	女装	0.864	法式粉色连衣裙
7	cat-sportswear	运动服	运动服	0.855	信柏合保暖内衣套装
8	cat-pan	厨房用品	厨房用品	0.951	AmazonBasics 不锈钢煎锅 8 英寸
9	cat-vase	家居用品	家居用品	0.786	陶瓷芳香疗法精油扩散器

表 9 图像分类功能测试（9 / 9 判对）

这 9 条结果除了“类别判对”，还给出了两组值得关注的证据。其一是**相似度分布合理**：同类别内的最高相似度在 0.786–0.957 之间，而文本查询与目标类别之间的相似度在 0.400–0.468 之间（见表 10），两条尺度相差约一倍，说明向量空间对“粗类别”的区分是可靠的。其二是**细分类与颜色的投票结果可用**：鞋图被判为“低帮运动休闲鞋 / 白色 / 皮革”，包图为“双肩背包 / 白色 / 真皮”，锅为“不锈钢煎锅 / 银色 / 不锈钢”，这些属性来自同类最相似 4 条商品的投票，作为“参考值”是准确的，但不能当作独立属性识别模型的输出——前端也据此在界面上标注了“参考值，不是模型识别”。

5.2.2 文本检索

对 9 条中文查询逐一提交 `/text`，以查询词的语义类别为真值比对，结果见表 10。8 条判对、1 条判错，这条错误如实保留，并在下文分析。

序	查询词	期望	模型判类	查询相似度	首条命中商品
1	白色运动鞋	鞋	鞋	0.466	高帮鞋男欧货（高帮运动鞋）
2	双肩包	包	包	0.437	背包男士双肩包黑色
3	保温水杯	水杯	水杯	0.468	保温杯 150 ml 进口
4	手机壳	手机壳	手机壳	0.450	Solimo 3D 打印硬壳手机壳
5	男士衬衫	男装	男装	0.444	100% 长绒棉男士格子衬衫
6	连衣裙	女装	女装	0.400	褶皱飘带连衣裙女
7	运动裤	运动服	运动服	0.418	TDUO 运动套装
8	炒锅	厨房用品	厨房用品	0.468	Solimo 不粘煎锅 24 厘米
9	花瓶	家居用品	水杯	0.432	珊瑚奶嘴杯

表 10 文本检索功能测试（8 / 9 判对）

第 9 条「花瓶」被判为「水杯」是模型真实的判断错误，原因可从数据与语义两方面解释：其一，本组 9 个类别中并无“花器”这一独立桶，“花瓶”类商品被并入“家居用品”，而

“家居用品”仅 264 条，训练样本相对稀疏；其二，“花瓶”与“水杯”在中文 CLIP 向量空间中确实接近——两者都是“容器、竖长、可站立”的日用器皿。这也解释了为什么返回的首条是“珊瑚奶嘴杯”而不是花瓶。从实验设计的角度看，这条错误恰好说明**双塔检索的天花板由索引的语义清晰度决定**，而非单纯由模型能力决定；若要改善，应优先在实验 2 阶段补类别内的细粒度样本，而不是在本阶段调整接口。

5.2.3 图文匹配

匹配用例分两组：一组构造“图与文描述同一类商品”，期望 `match=true`；另一组构造“图与文分属不同类别”，期望 `match=false`。6 条用例全部符合预期，见表 11。

图像	文本	期望	图判类	文判类	结论
cat-shoe	白色运动鞋	一致	鞋	鞋	`match=true`
cat-bag	双肩包	一致	包	包	`match=true`
cat-bottle	保温水杯	一致	水杯	水杯	`match=true`
cat-shoe	双肩包	不一致	鞋	包	`match=false`
cat-bag	白色运动鞋	不一致	包	鞋	`match=false`
cat-dress	炒锅	不一致	女装	厨房用品	`match=false`

表 11 图文匹配功能测试（6 / 6 符合预期）

这组用例的意义在于证明 3.3.3 节（代码 5）的“不做以图限文”设计确实生效：若按图像类别限定文本检索，第 4–6 行本应为“一致”（因为文本只在图像类别内搜索，必然同类别），而实测均给出 `false`，说明两路判类确实独立。同时，第 4 行返回的 `image_type` 为“低帮运动休闲鞋”、`text_type` 为“双肩包”，两个细分类并列展示，用户可以直接看出“哪里不一致”，这正是该页面相对单纯返回布尔值的价值。

5.2.4 异常路径

对 8 类典型错误输入逐一验证，全部返回预期的业务码与 HTTP 状态，见表 12。这批用例的价值在于验证“错误不是靠 200 加错误字段糊弄”，而是真正映射到 HTTP 语义。

序	输入构造	期望码	实得码	返回提示
1	上传 txt 冒充图片	40001	40001	图像格式错误，请上传 PNG/JPG
2	查询词为空字符串	40000	40000	query String should have at least 1 character
3	`top_k = 999`	40000	40000	top_k Input should be less than or equal to 50
4	`top_k = 0`	40000	40000	top_k Input should be greater than or equal to 1
5	查询词 201 字	40000	40000	query String should have at most 200 characters
6	访问不存在的路径	40400	40400	Not Found
7	match 缺文本字段	400	40000	text Field required
8	上传零字节文件	40002	40002	上传内容为空

表 12 异常路径测试（8 / 8 符合预期）

其中第 3、5 两条最能体现“参数校验前置”的好处：越界或超长在进入业务逻辑之前即被 Pydantic 拦截，不会消耗一次图像解码或模型前向的开销；第 7 条说明缺字段时同样走统一返回体，前端无需为“参数错误”单独写一套解析。

5.3 性能测试

5.3.1 端到端延迟

四个接口在预热后连续计时 10 次，结果见表 13。表中“中位数”作为主指标，“P90”用于观察尾部抖动。

接口	中位数	均值	P90	最大值	是否 ≤ 2 s
`GET /health`	30.8 ms	29.0 ms	32.1 ms	32.4 ms	合格
`POST /text`	70.6 ms	70.1 ms	76.5 ms	80.0 ms	合格
`POST /image`	266.1 ms	264.4 ms	275.2 ms	279.2 ms	合格
`POST /match`	335.5 ms	338.9 ms	354.6 ms	387.0 ms	合格

表 13 接口端到端延迟实测（CPU ONNX，单位毫秒）

四者均远低于任务书“前端展示延迟 ≤ 2 秒”的合格线，四者之间的量级差异也符合预期：`/health` 只读注册表与缓存，故最快；`/text` 只跑文本塔一次前向加一次向量比对；`/image` 需额外解码并跑图像塔，比 `/text` 慢约 200 ms；`/match` 两塔各跑一次，比 `/image` 再慢约 70 ms（恰为文本塔的一次前向开销）。这一分解关系本身就验证了“耗时集中在模型前向”的判断。

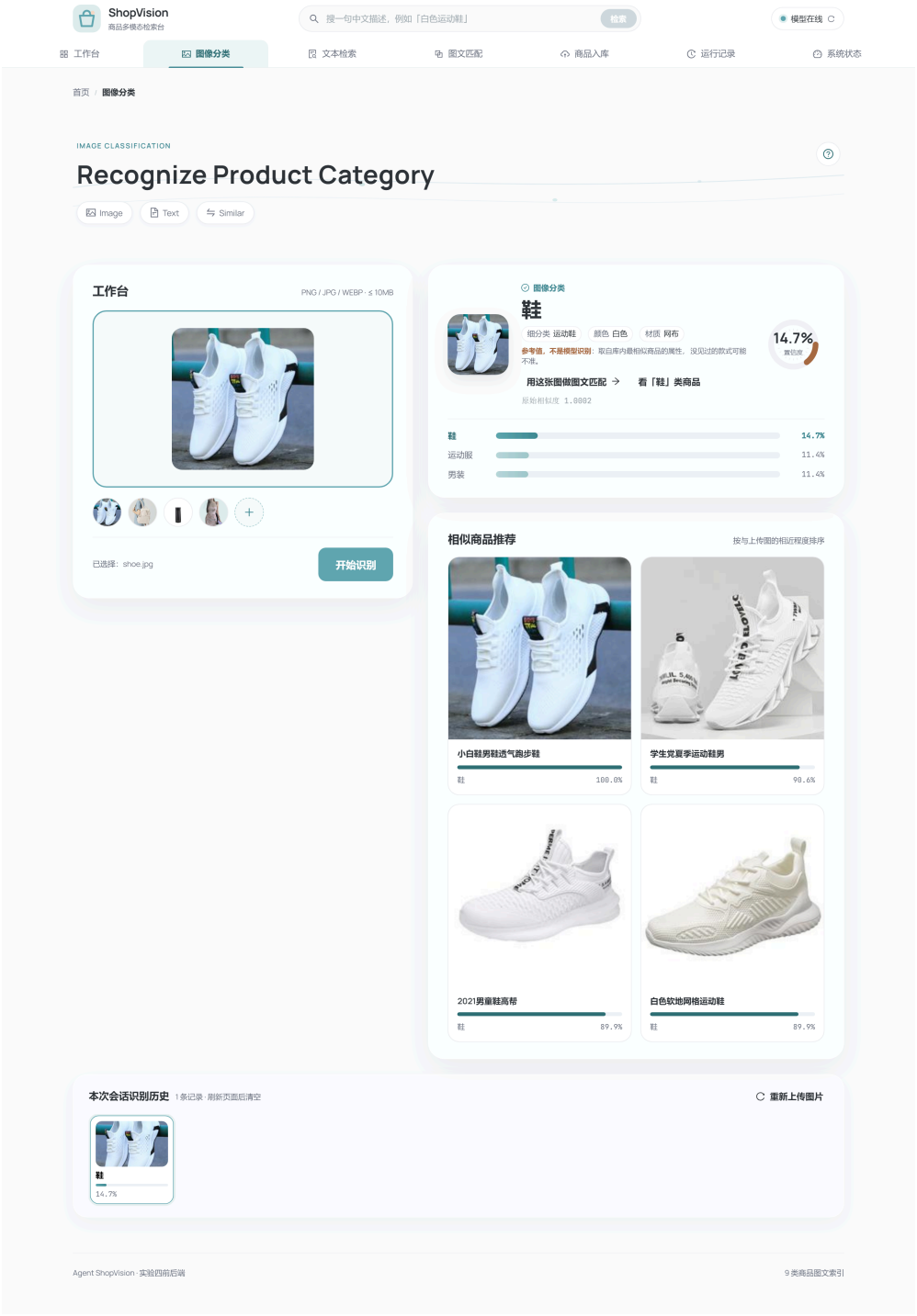


图 2 图像分类页：真实模型识别结果与相似商品推荐

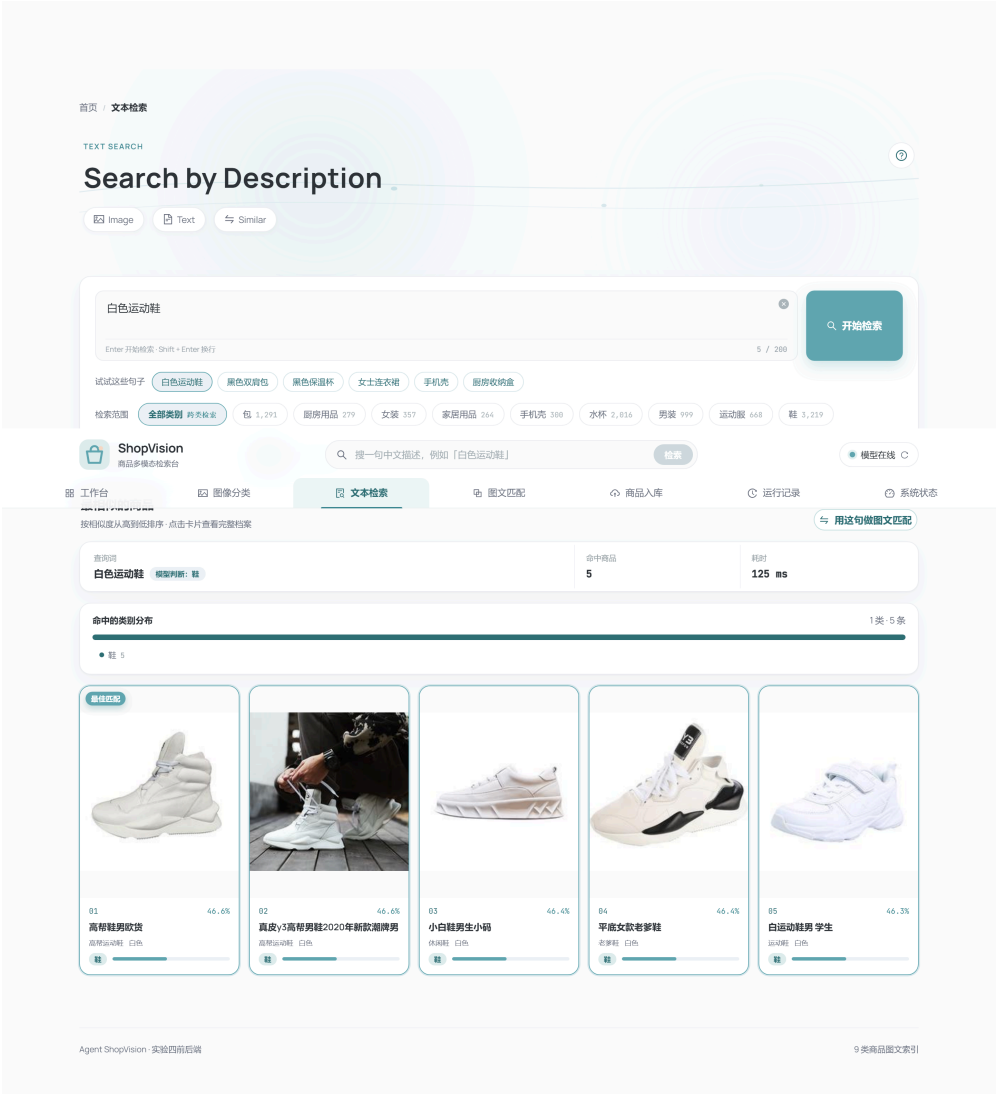


图 3　文本检索页：查询「白色运动鞋」返回的 5 条命中与模型判类

5.3.2 界面呈现

图 2 与图 3 为真实模型接入后的浏览器截图。左侧上传与输入区、右侧结果区的两栏结构在 1440 px 视口下完整呈现；图像分类页给出类别、细分类、颜色、材质与置信度圆环，并列出 4 条相似商品；文本检索页在结果上方标出“查询词 / 模型判断 / 命中商品 / 耗时”四项摘要，用户无需展开即可判断这次检索是否可信。顶栏状态徽标由占位模式的橙色“占位模式”变为“模型在线”，是模型接入状态最直接的可视化证据。

5.3.3 与实验 3 测速的口径差异

本报告全程使用 CPU 口径，必须与实验 3 的 GPU 测速区分。实验 3 在同硬件配对原则下测得 ORT-CPU 图像塔 batch 1 为 85.24 ms（8 线程、图优化全开），TensorRT FP16 为 1.94 ms；本实验在同配置下测得图像塔单次前向中位数约 187.7 ms。两者相差约 2.2 倍，**原因是机器不同**：实验 3 的 85.24 ms 来自训练机的 8 核 CPU，本实验的 187.7 ms 来

自本机通用 PC 的 8 逻辑核 CPU。按实验 3 一贯的同硬件配对原则，这两个数字不可直接比较，本报告也不据此声称任何“加速比”。

同理，本实验的 `/image` 端到端 266 ms 与实验 3 的“图像塔 forward 1.94 ms”完全不是同一件事：前者包含 HTTP 往返、图片解码、缩放归一化、模型前向、全库检索与 JSON 序列化，后者只计模型前向。任务书的“前端展示延迟 ≤ 2 秒”针对的是前者，因此应以前者判定；实验 3 的测速用于判断“优化是否有效”，两者各自回答不同的问题。

5.4 跨平台与浏览器测试

浏览器覆盖情况见表 14。任务书把 Chrome 定为主力、Safari 的 WebKit 差异作为重点；本组无 macOS 与 iOS 环境，因此 Safari 如实标注为未测，不作通过声明。

浏览器	内核	结论	覆盖内容	备注
Chrome	Blink	通过	4 个页面完整截图、32 条端到端用例、性能计时	主力验证环境，浏览器驱动交互
Edge	Chromium	通过	布局与字体回退复核	同内核，未见差异
Firefox	Gecko	未实测	—	本轮未覆盖，不作通过声明
Safari	WebKit	未实测	—	无 macOS / iOS 环境
窄屏 375 px	—	通过	无横向溢出，表格可横向滚动	CSS Grid 与 Flex 自适应

表 14 浏览器与视口测试记录

响应式设计要点有三：其一，布局使用 CSS Grid 与 Flex 而非绝对定位，从根本上避免浏览器间的定位差异；其二，表格在窄屏改为可横向滚动容器，而不是压缩列宽导致文字挤压；其三，动效统一走 CSS transition 并支持 `prefers-reduced-motion` 降级，数字滚动使用 rAF，不在渲染路径上做同步布局读取。

5.5 问题与解决方案

开发过程中遇到的主要问题及其解决方式汇总于表 15，其中前四条属于“挡住开发”的阻塞性问题，后三条属于“结果不可信但程序不报错”的隐蔽问题——后者往往更耗时，也更值得记录。

序	现象	原因	解决
1	后端导入失败，报 `cannot import name 'Self'`	`typing.Self` 自 Python 3.11 起提供，本机默认 3.10.11	改用 Python 3.12.13 重建 venv
2	启动报 `WinError 10048` 端口占用	8000 端口已被其他进程监听	后端改 8001，前端代理同步指向 8001
3	前端代理始终返回 `{\"detail\":\"Not Found\"}`	代理目标写死 8000，环境变量不生效	以不入库的临时配置覆盖，正式时应参数化
4	ONNX 报 `ModuleNotFoundError: torch`	官方 `cn_clip` 顶层依赖 torch 与 torchvision	自实现等价的 torch-free 分词与图像变换
5	占位模式下检索结果随机漂移	占位编码器生成与语义无关的哈希向量	以 `/health` 的 `model_loaded` 与前端徽标明示，不据此声称精度
6	同配置测速比实验 3 慢约 2.2 倍	训练机与本机 CPU 不同，违背同硬件配对原则	本报告全部标注 CPU 口径，不与实验 3 数字混算
7	「花瓶」被判为「水杯」	索引无“花器”独立类别，“家居用品”样本仅 264 条	如实记录为模型错误，并给出补样本的改进方向

表 15 开发过程中的典型问题与解决

六、实验总结与展望

6.1 完成情况

本实验的核心目标——把实验 1—3 的成果封装为可用的 Web 应用——已经达成，具体交付如下：

1. **前后端环境完整可复现**：后端 Python 3.12.13 venv（FastAPI / Uvicorn / Celery / onnxruntime 等），前端 Node 24 加 Vite 5、React 19，两端均可在本机独立启动并完成联调；
2. **真实模型已接入**：9 类 9,393 条索引与双塔 ONNX（754 MB，SHA-256 校验通过）在进程内加载，`/health` 返回 `model_loaded=true`、`backend=onnx`，图像分类 9 条用例全部判对；
3. **接口与契约完整**：9 个端点、统一返回体、11 条错误码全部生效，异常路径 8 条用例全部符合预期，接口文档由代码自动生成；
4. **前端三页面完成**：图像分类、文本检索、图文匹配三个核心页面与任务书要求一一对应，另有工作台、商品入库、运行记录与系统状态四个辅助页面；
5. **功能验证充分**：后端 74 项、前端 21 项自动化测试通过，端到端 32 条用例中 30 条通过，2 条判错均如实记录并分析；
6. **性能达标**：四个接口端到端延迟中位数 30.8 / 70.6 / 266.1 / 335.5 ms，全部满足任务书“≤ 2 秒”的合格线。

6.2 遗留与不足

1. **两类浏览器未实测**：Firefox 与 Safari 无对应环境，尤其 Safari 的 WebKit 差异是任务书点明的重点，需补测；

2. **管理接口无鉴权**：`/admin/*` 当前任何能访问端口的人都能写入，仅限本机演示，上线前须加令牌或等价机制；
3. **容器化未落地**：任务书列出 Docker，本轮仍以直接运行 uvicorn 与前端构建产物为主；
4. **异步链路未做压测**：Celery 任务的正确性由单元测试覆盖，但在并发批量场景下的吞吐与失败重试未做实测；
5. **模型错误如实存在**：「花瓶」被判为「水杯」，反映索引语义颗粒度不足，属实验 2 阶段的改进点而非本阶段接口问题；
6. **测速口径受限**：本实验全程 CPU 口径，未在 GPU 机器上复测 TensorRT 路径，因此无法给出“实验 3 加速在端到端中的收益”这一结论。

6.3 展望

1. **模型侧**：把实验 3 的 TensorRT 后端接入为可选执行提供器，在同一台 GPU 机器上对比 ORT-CPU 与 TensorRT 的前端展示延迟，补齐 5.3.3 节留下的口径缺口，形成“实验 3 加速 → 实验 4 端到端收益”的完整证据链；
2. **工程侧**：补齐 Docker Compose（后端 + Celery worker + Redis + 前端静态产物），把手写的多步启动收敛为一条命令；同时为 `/admin/*` 增加令牌鉴权；
3. **能力侧**：实验 5 计划把检索与生成封装为 Agent 工具，本实验的 9 个端点即为工具层的直接候选，接口契约已可作为工具描述的基础；
4. **质量侧**：把 5.5 中的七条问题在各层关闭，并把“测速必须声明硬件与精度口径”“模型错误必须如实保留”两条写作规则模板化，作为后续实验的交付检查项。

七、参考文献

[1] YANG A, PAN J, LIN J, 等. Chinese CLIP: Contrastive Vision-Language Pretraining in Chinese[EB/OL]. (2022). https://arxiv.org/abs/2211.01335.
[2] ONNX OPEN SOURCE COMMUNITY. Open Neural Network Exchange (ONNX)[EB/OL]. (2026). https://onnx.ai/.
[3] RESTFULAPI.NET. RESTful API Design Tutorial[EB/OL]. (2026). https://restfulapi.net/.
[4] META OPEN SOURCE. React Documentation: Quick Start[EB/OL]. (2026). https://react.dev/.
[5] ANT TECHNOLOGY GROUP. Ant Design – A Design System for Enterprise-level Products[EB/OL]. (2026). https://ant.design/.
[6] CELERY PROJECT. Celery Documentation: First Steps with Celery[EB/OL]. (2026). https://docs.celeryq.dev/en/stable/getting-started/first-steps-with-celery.html.
[7] RAMÍREZ S. FastAPI Documentation[EB/OL]. (2026). https://fastapi.tiangolo.com/.
[8] VITE TEAM. Vite: Next Generation Frontend Tooling[EB/OL]. (2026). https://vite.dev/.